

PEREGRINE Flight Stack

Architecture, Design & Multi-Agent Integration

April 7, 2026

Agenda

Part I — Design & Architecture

1. Project Overview & Goals
2. Layered Architecture
3. Data Flow & Timing
4. Frame Convention Strategy
5. Plugin Architecture
6. Safety Architecture
7. UAV Manager — State Machine

Part II — Multi-Agent & Deployment

8. Multi-Agent Architecture
9. Containerized Deployment
10. Monitoring & Visualization

Part III — Code Overview

11. Interfaces & Messages
12. Key Package Walk-through
13. Implementation Status

Appendix — ROS 2 & PX4 Background

Part I: Design & Architecture

What is PEREGRINE?

PEREGRINE — *Aerial Robotics Infrastructure for Operational Navigation*

A ROS 2-based autonomous flight stack for **multi-agent UAV operations** with PX4 autopilots.

Key Characteristics

- Manager-based modular architecture
- Plugin-based algorithms (swap at runtime)
- Multi-platform: simulation, Jetson Orin, RPi 5
- Multi-agent with collision avoidance
- Safety-first with layered monitoring
- Fully containerized (Docker)

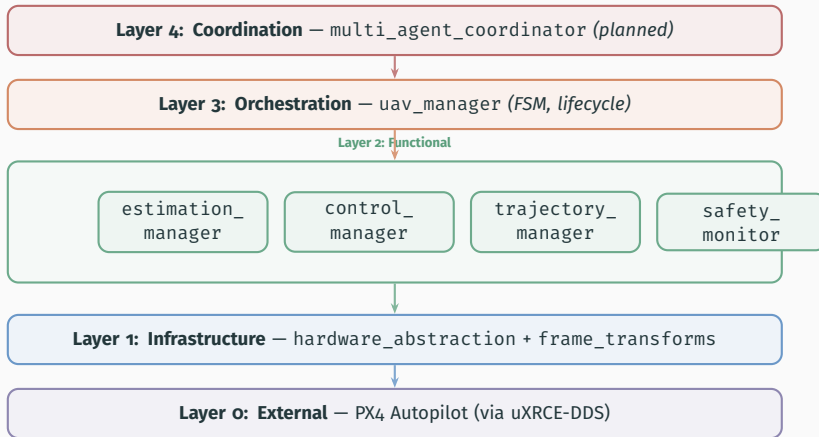
Technology Stack

Language	C++17
Middleware	ROS 2 Humble / Jazzy
Autopilot	PX4 v1.15+ (uXRCE-DDS)
Simulation	Gazebo Harmonic, PX4 SITL
Math	Eigen 3
Plugins	ROS 2 pluginlib
Containers	Docker + Compose
Inter-container	Zenoh bridge

Design Goals

Goal	Approach
Modularity	Each functional domain is a separate ROS 2 package with clean interfaces
Extensibility	Plugin architecture — add new algorithms without modifying core
Safety	Three-layer safety: manager checks, safety monitor, PX4 hardware failsafes
Multi-agent	Namespace isolation, Buffered Voronoi Cell collision avoidance
Deployability	Docker images per platform, systemd for hardware deployment
Research-friendly	Swap controllers / estimators at runtime via service calls

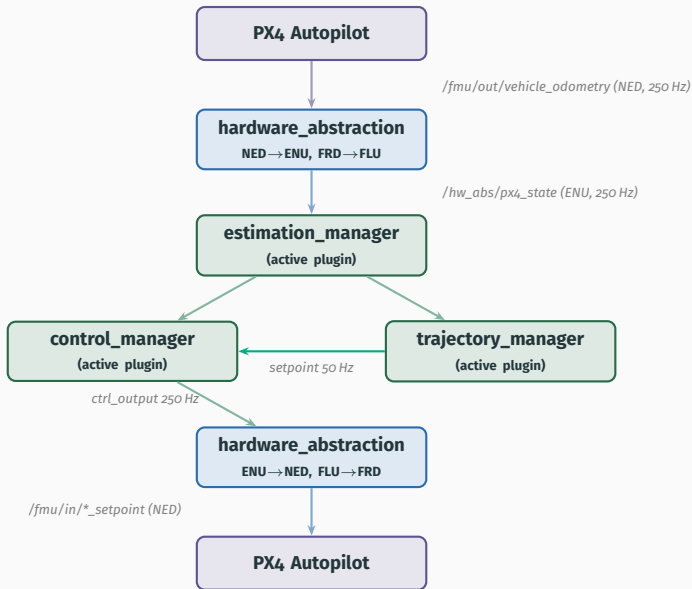
Layered Architecture



Package Map

Package	Type	Layer	Purpose
peregrine_interfaces	Interfaces	Cross	All custom msgs, srvs, actions
frame_transforms	Library	1	ENU/NED conversions, TF2 broadcasting
hardware_abstraction	Node	1	Sole PX4 interface, frame conversion boundary
estimation_manager	Node + Plugins	2	State estimation plugin management
control_manager	Node + Plugins	2	Flight controller plugin management
trajectory_manager	Node + Plugins	2	Trajectory generation and evaluation
safety_monitor	Node	2	Geofence, heartbeats, envelope protection
uav_manager	Node	3	FSM, lifecycle, takeoff/land sequences
tui_status	Node	Tool	ncurses real-time monitoring display
peregrine_bringup	Launch/Config	Deploy	Launch files, YAML configs

Data Flow — Control Loop



Timing Requirements

Loop	Frequency	Deadline	Notes
State estimation	250 Hz	4 ms	Matched to PX4 odometry rate
Control output	250 Hz	4 ms	Must be deterministic
Trajectory evaluation	50 Hz	20 ms	Setpoints interpolated at controller rate
Safety monitoring	100 Hz	10 ms	Highest priority — independent watchdog
Offboard heartbeat	≥ 2 Hz	500 ms	PX4 requirement — exits offboard if missed
TUI display	10 Hz	100 ms	Non-critical

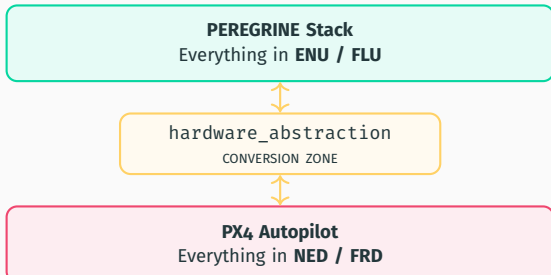
Frame Convention Strategy

The Problem: PX4 uses NED/FRD, ROS 2 uses ENU/FLU.

Mixing them is the #1 source of “drone flew the wrong way” bugs.

Golden Rule: All PEREGRINE packages operate in **ENU/FLU**.

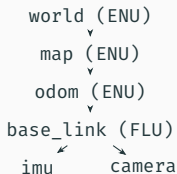
Conversion happens at *exactly one point*:
hardware_abstraction.



Conversion Summary

Qty	ENU→NED	FLU→FRD
Position	$(y, x, -z)$	$(x, -y, -z)$
Quaternion	$(w, y, x, -z)$	same
Yaw	$\frac{\pi}{2} - \psi_{\text{enu}}$	N/A
Covariance	$R C R^T$	same

TF2 Frame Tree (REP 105/103)



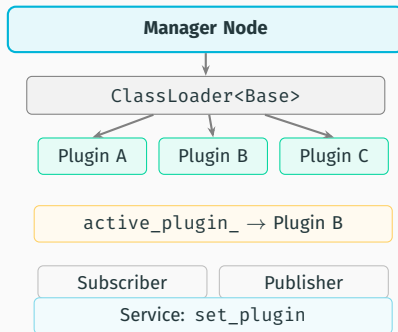
Plugin Architecture

Design Rationale

	Without	With Plugins
New algo	Modify core, recompile	Compile plugin, load via YAML
Runtime switch	Impossible	Service call
Testing	Full system	Mock interface

Plugin Lifecycle (common to all managers)

1. `initialize(node, name)` – receive parent node
2. `activate()` – start processing
3. `deactivate()` – stop processing
4. `reset()` – clear internal state



Three Plugin Domains

Domain	Base Class	Input	Output	Plugins
Estimation	EstimatorBase	PX4 state, ext. pose	State	PX4 Passthrough
Control	ControllerBase	State + Setpoint	ControlOutput	PX4 Passthrough, SE(3)
Trajectory	TrajGeneratorBase	Waypoints / goals	TrajSetpoint	Waypoint, Hold, Takeoff/Land

Control Hierarchy — Where PEREGRINE Meets PX4

PEREGRINE computes

POSITION: pos setpoint
VELOCITY: vel setpoint
ATTITUDE: quat + thrust
BODY_RATE: rates + thrust

PX4 handles the rest

→ Pos → Vel → Att → Rate → Motors
→ Vel → Att → Rate → Motors
→ Att → Rate → Motors
→ Rate → Motors

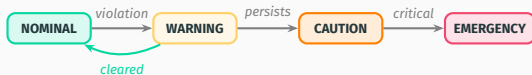
Safety Architecture — Three Layers

Layer 3: PX4 Hardware Failsafes (last resort)

Layer 2: PEREGRINE Safety Monitor

Layer 1: Manager-Level Checks

Safety Status State Machine



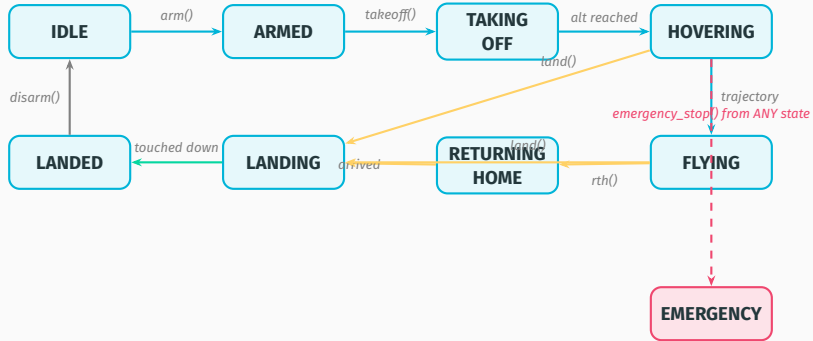
Safety Checks

Check	Rate	Action
Geofence	100 Hz	Land/RTH
Heartbeat	Cont.	E-land/RTH
State valid.	100 Hz	Hover→Land
Envelope	100 Hz	Limit output
Battery	10 Hz	Warn→RTH→Land
Collision	20 Hz	Slow→Stop

Heartbeat Priorities

Source	Priority	Timeout	Loss
hw_abstraction	Critical	1.0 s	E-land
estimator_mgr	Critical	0.5 s	E-land
control_mgr	High	0.5 s	RTH
GCS	High	5.0 s	RTH
trajectory_mgr	Medium	1.0 s	Hold

UAV Manager — State Machine



Preflight Checks & Takeoff Sequence

Preflight Checks (before arming)

1. `hardware_abstraction` connected to PX4
2. `estimation_manager` healthy & publishing
3. `control_manager` has active controller
4. `safety_monitor` reports NOMINAL
5. Battery above minimum threshold
6. GPS fix adequate (outdoor) or MoCap (indoor)
7. Geofence loaded and valid

Takeoff Sequence

1. Assert state = ARMED, run preflight checks
2. Transition to TAKING_OFF
3. Request takeoff trajectory (target altitude)
4. Command PX4 offboard mode
5. **Loop:** publish feedback (altitude, progress%)
 - If safety alert or timeout → abort & land
6. Altitude reached → transition to HOVERING

Startup Order

`hw_abstraction` → `estimation_mgr` → `control_mgr` →
`trajectory_mgr` → `safety_monitor` → `uav_manager`

Part II: Multi-Agent & Deployment

Multi-Agent Architecture — Overview

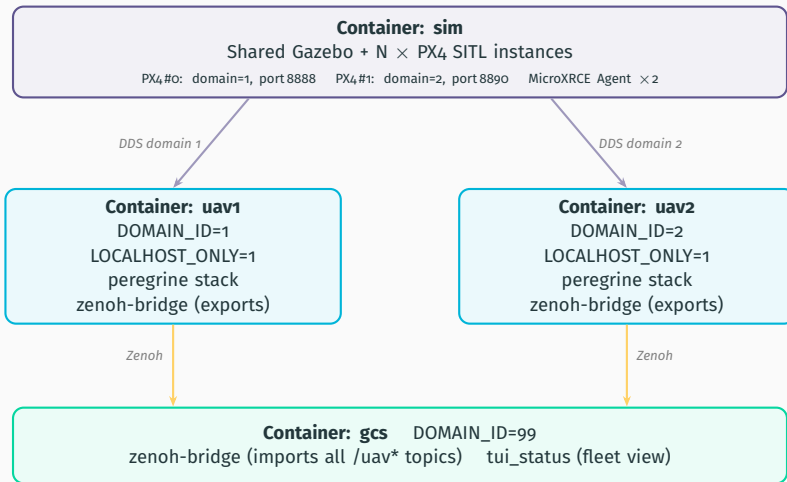
Implementation Status

Component	Status
Per-UAV namespace isolation	Done
DDS domain container isolation	Done
Zenoh inter-container bridging	Done
Multi-instance PX4 SITL	Done
GCS TUI multi-UAV view	Done
BVC collision avoidance	Planned
Consensus protocols	Planned
Decentralized coordination	Planned

Key Design Decisions

- **DDS domain isolation:** each UAV container has its own `ROS_DOMAIN_ID`
- **ROS_LOCALHOST_ONLY=1:** DDS stays within each container
- **Zenoh bridges:** selectively export only status topics (8 per UAV)
- **network_mode:** **host:** all containers share host network for Zenoh
- **GCS domain 99:** isolated, receives only bridged status — prevents accidental command injection

Multi-Container Architecture



Zenoh Bridge — Selective Topic Export

Bridged Topics per UAV (8 total)

1. `uav_state` — raw PX4 state
2. `estimated_state` — filtered state
3. `safety_status` — safety monitor output
4. `estimation_status` — estimator health
5. `control_status` — controller health
6. `trajectory_status` — trajectory progress
7. `status` — comprehensive UAV status
8. `gps_status` — GPS fix quality

Why Zenoh?

- `ROS_LOCALHOST_ONLY` prevents DDS cross-container leaks
- Zenoh provides *selective, explicit* topic sharing
- Status-only bridging: GCS observes but *cannot inject commands*
- Peer-to-peer discovery — no central broker needed
- Low-overhead protocol, works across networks

GCS Bridge

Subscribes to `uav*/` prefix and republishes on its local DDS domain (99).

Planned: Buffered Voronoi Cell Collision Avoidance

Concept

Each UAV computes its own *Voronoi cell* — the region closer to itself than to any neighbor. The cell is shrunk inward by a safety buffer. If each UAV stays within its buffered cell, **minimum separation is guaranteed**.

Algorithm (per UAV i)

For each neighbor j :

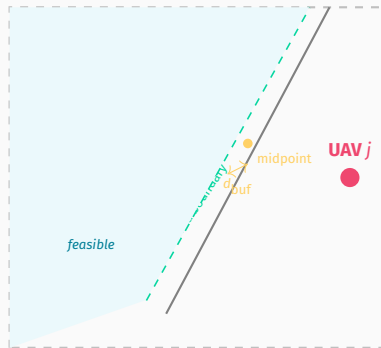
$$\mathbf{m} = \frac{1}{2}(\mathbf{p}_i + \mathbf{p}_j)$$

$$\mathbf{n} = \text{normalize}(\mathbf{p}_i - \mathbf{p}_j)$$

$$\mathbf{b} = \mathbf{m} + \mathbf{n} \cdot d_{\text{buffer}}$$

Constraint: $\mathbf{n} \cdot \mathbf{x} \leq \mathbf{n} \cdot \mathbf{b}$

Constraints published $\rightarrow$ trajectory_manager
projects setpoints onto feasible region.



Containerized Deployment

Docker Image Strategy

Image	Platform	Features
simulation	x86 + NVIDIA	CUDA, Gazebo, PX4 SITL
jetson	arm64	CUDA, TensorRT
rpis	arm64	Lightweight, headless

All images:

- Mount full repo workspace → /ros2_ws
- Non-root user mapped to host UID/GID
- DRONE_ID → ROS_DOMAIN_ID
- Optionally start MicroXRCE Agent

Repository Structure

```
peregrine/  
  src/  
    peregrine_core/ (11 pkgs)  
    px4_msgs/ (submodule)  
    peregrine_app_*/ (future)  
  docker/  
    docker/, compose/, config/  
    Makefile  
  docs/
```

Operational Commands

make build-sim	Build sim image
make dev	Dev shell (disposable)
make multi-sitl	2-UAV sim + Zenoh
DRONE_ID=1 make jetson	Run on Jetson

Monitoring — TUI Status Display

debug_TUI.png

ncurses Terminal UI

Real-time monitoring at 10 Hz

Features

- Color-coded: **green** (nominal), **yellow** (warning), **red** (emergency)
- Multi-UAV compact fleet view
- Keyboard navigation between UAVs
- Alert ring buffer (100 entries)
- Non-blocking input

Display Sections

- Flight state, mode, armed status
- Position / velocity (ENU)
- Active manager plugins
- Safety status, geofence distance
- Connection health (heartbeats)
- Battery voltage / percent

6-uav_rviz.png

RViz2 Flight Visualization

- 6-UAV swarm visualization
- Per-UAV coordinate frames via TF2
- Real-time position and orientation
- Trajectory path display
- Multi-UAV markers with namespace isolation
- REP 105 frame tree:
world → map → odom →
base_link

Part III: Code Overview

Custom Interfaces — peregrine_interfaces

Design Principles

- **Single package** for all .msg, .srv, .action
- Semantic field names with units: altitude_m, velocity_mps
- Frame-explicit: all spatial msgs include frame_id
- All messages timestamped via std_msgs/Header

Core: State.msg

- Position (ENU), Velocity, Acceleration
- Orientation (quaternion ENU→FLU)
- Angular velocity (FLU body) [rad/s]
- 6×6 pose & twist covariance
- Estimator name + confidence [0-1]

ControlOutput.msg

- control_mode: POS / VEL / ATT / BODY_RATE
- Mode-dependent fields; thrust_normalized [0-1]

TrajectorySetpoint.msg

- Validity flags per field
- Position, velocity, acceleration, jerk, yaw, yaw_rate

SafetyStatus.msg

- Safety level (NOMINAL → EMERGENCY)
- Component health flags, geofence status
- Heartbeat counts, battery, active alerts
- Recommended action

Actions

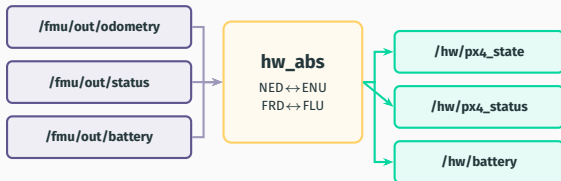
- Takeoff, Land, SetTrajectory, GoToPosition

Services

- SetController, SetEstimator, Arm, EmergencyStop

Hardware Abstraction — Sole PX4 Interface

Bidirectional Translation



Key Responsibilities

- Frame conversion at the boundary
- PX4 QoS matching (best-effort, transient-local)
- Offboard mode heartbeat (≥ 2 Hz)
- NaN convention for unused fields
- Time sync (PX4 μ s $\leftrightarrow$ ROS 2 ns)
- Arm/disarm service
- Control output $\rightarrow$ PX4 setpoint topic

Critical: Wrong QoS = *silent* comm failure.
PX4 requires best-effort + transient-local.

SE(3) Geometric Controller

Geometric Tracking on SE(3)

Lee, Leok, McClamroch — “Geometric Tracking Control of a Quadrotor UAV on SE(3)”

Algorithm:

1. Position error: $\mathbf{e}_p = \mathbf{p} - \mathbf{p}_d$
2. Velocity error: $\mathbf{e}_v = \mathbf{v} - \mathbf{v}_d$
3. Desired accel: $\mathbf{a}_d = -K_p \mathbf{e}_p - K_d \mathbf{e}_v + \mathbf{a}_{ff} - g \hat{\mathbf{z}}$
4. Desired thrust dir: $\hat{\mathbf{z}}_d = \text{normalize}(\mathbf{a}_d)$
5. Thrust magnitude: $F = m (\mathbf{a}_d \cdot \hat{\mathbf{z}}_b)$
6. Construct R_d from $\hat{\mathbf{z}}_d$ + desired yaw
7. **Output:** desired quaternion + normalized thrust

Output mode: ATTITUDE $\rightarrow$ PX4 attitude controller handles rate + motors

Configurable Gains

Parameter	Default
K_p (position)	[6.0, 6.0, 8.0]
K_d (velocity)	[4.5, 4.5, 5.0]
$K_{p,att}$	[8.0, 8.0, 2.0]
$K_{d,att}$	[1.5, 1.5, 0.4]
Mass	1.5 kg
Max thrust	30.0 N

Adding a Custom Controller

1. Implement ControllerBase
2. Register with PLUGINLIB_EXPORT_CLASS
3. Add to config YAML
4. Switch at runtime via service

Available Generators

Generator	Output	Use Case
WaypointLinear	POS+YAW	Linear interpolation
HoldPosition	POS+YAW	Hover, e-hold
TakeoffLand	POS+YAW	Vertical w/ S-curve
Polynomial	FULL	Min-snap (planned)

Setpoint Loop (50 Hz)

1. Compute elapsed = $t_{\text{now}} - t_{\text{start}}$
2. Evaluate setpoint from generator
3. Check BVC collision constraints
4. If violating $\rightarrow$ project onto boundary
5. Publish setpoint; mark completed when done

Action Interface

SetTrajectory: goal (trajectory + replace flag), feedback (progress %, error), result (success/fail)

GoToPosition: goal (target ENU, yaw, velocity), feedback (distance, ETA), result (success/fail)

Constraint Integration

BVC half-plane constraints from multi-agent coordinator are applied during setpoint evaluation. Violating setpoints are projected onto the nearest feasible point.

Implementation Status

Package	Status	Notes
peregrine_interfaces	Done	All messages, services, actions defined
frame_transforms	Done	Conversions + TF broadcaster, unit tested
hardware_abstraction	Done	PX4 bridge, all control modes, time sync
estimation_manager	Done	Manager + PX4 passthrough plugin
control_manager	Done	Manager + PX4 passthrough + SE(3) geometric
trajectory_manager	Done	Waypoint linear + hold + takeoff/land
safety_monitor	Done	Geofence, battery, envelope, GPS, rule engine
uav_manager	Done	State machine, preflight, takeoff/land
tui_status	Done	Single + multi-UAV views, ncurses
peregrine_bringup	Done	Launch files, configs, multi-UAV SITL
Docker stack + Zenoh	Done	Sim/Jetson/RPi5 images, inter-container bridging
multi_agent_coord.	Planned	BVC, consensus, decentralized coordination
Polynomial trajectories	Planned	Minimum-snap trajectory generation
Custom EKF estimator	Planned	Multi-sensor fusion

Summary

What PEREGRINE Provides

- Modular, plugin-based flight stack
- Clean PX4 integration with single conversion boundary
- Three-layer safety architecture
- Multi-agent support with container isolation + Zenoh
- Research-friendly: swap algorithms at runtime
- Multi-platform deployment (sim, Jetson, RPi5)

Next Steps

- BVC collision avoidance implementation
- Minimum-snap polynomial trajectories
- Custom EKF for multi-sensor fusion
- Decentralized coordination protocols
- Hardware flight testing campaign
- Distributed task allocation

Questions?

Appendix: ROS 2 & PX4 Background

A.1 — What is ROS 2?

- Robot Operating System 2 — middleware framework (not an OS)
- Built on DDS (Data Distribution Service) for real-time distributed communication
- Language support: C++, Python (PEREGRINE uses **C++17**)

Core Communication Primitives

Primitive	Pattern	Blocking?	Use Case
Topics	Pub/Sub (many-to-many)	No	Sensor streams, state estimates, control outputs
Services	Request/Response (1:1)	Yes	Config changes, one-shot commands
Actions	Goal/Feedback/Result	No	Long-running: takeoff, trajectory execution

A.2 — ROS 2 QoS & Execution Model

Quality of Service (QoS)

Profile	Reliability	Use Case
Sensor Data	Best Effort	High-rate streams
Parameters	Reliable + TL	Config, late join
Default	Reliable	Services, actions

- **Best Effort:** drop if subscriber can't keep up
- **Reliable:** retransmit until received
- **Transient Local:** late joiners get last value
- QoS mismatch = *silent* communication failure

Executors & Callback Groups

- **Node:** smallest computation unit
- **Executor:** dispatches callbacks to threads
- **SingleThreaded:** one callback at a time
- **MultiThreaded:** concurrent (PEREGRINE uses this)
- **MutuallyExclusive** group: implicit mutex
- **Reentrant** group: user manages thread safety

Lifecycle (Managed) Nodes

- Unconfigured → Inactive → **Active** → Inactive
- Enables coordinated startup ordering
- PEREGRINE managers use lifecycle + auto_start

A.3 — ROS 2 Namespaces & pluginlib

Namespaces for Multi-Robot

Single UAV	Multi-UAV
/hw_abs/px4_state	/uav1/hw_abs/px4_state
/est_mgr/state	/uav1/est_mgr/state
	/uav2/hw_abs/px4_state
	/uav2/est_mgr/state

- ROS 2 namespaces isolate per robot
- DDS Domain IDs for network-level isolation
- Combined with LOCALHOST_ONLY for containers

pluginlib — Runtime Plugin Loading

- Load C++ classes at runtime from shared libraries
- Base class defines interface, plugins implement
- Registration: XML descriptors + `PLUGINLIB_EXPORT_CLASS`
- No recompilation of manager for new algorithms

Launch System

- Python-based launch files
- Supports arguments, conditionals, environment variables
- Include other launch files for modularity

A.4 — PX4 Autopilot Overview

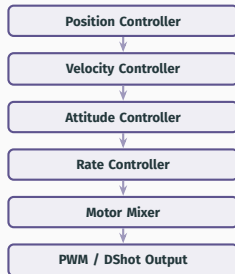
What is PX4?

- Open-source autopilot for drones, VTOL, rovers
- Low-level flight control: attitude stabilization, motor mixing, failsafes
- Runs on dedicated hardware (Pixhawk, etc.)
- PEREGRINE interfaces as companion computer stack

Offboard Mode

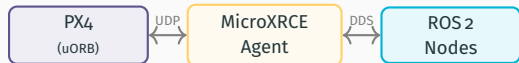
- PX4 accepts external commands from companion
- Must publish `OffboardControlMode` ≥ 2 Hz
- Start publishing *before* requesting mode switch
- Setpoint stream determines control level

PX4 Control Cascade



A.5 — uXRCE-DDS Bridge & Frame Conventions

uXRCE-DDS Bridge

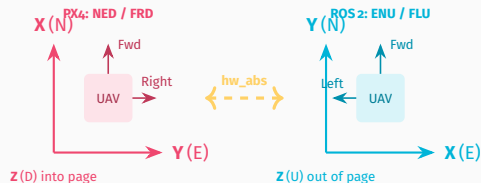


- Replaces MAVLink for ROS2 integration
- PX4: /fmu/out/* (from), /fmu/in/* (to)

Key Points

- Frame mismatch = #1 source of direction bugs
- PEREGRINE: all internal in ENU/FLU
- Single conversion in hardware_abstraction

PX4 (NED/FRD) vs ROS 2 (ENU/FLU)



	World	Body
PX4	NED: $X=N, Y=E, Z=D$	FRD: Fwd, Right, Down
ROS 2	ENU: $X=E, Y=N, Z=U$	FLU: Fwd, Left, Up

A.6 — PX4 SITL & Configuration

PX4 Software-In-The-Loop

- Full PX4 autopilot on host machine
- Connects to Gazebo Harmonic for physics
- Identical code to what runs on hardware
- Multi-instance: N PX4 instances with different ports/domain IDs

PEREGRINE Config Structure

```
config/  
  default.yaml  
  environments/  
    gps.yaml, mocap.yaml  
  uav_types/  
    quadrotor_x.yaml  
  fleet/  
    4_uav_diamond.yaml
```

Key Default Parameters

Node	Parameter	Value
hw_abstraction estimator_mgr	offboard_rate_hz	10.0
	publish_rate_hz	250.0
	default_estimator	px4_passthrough
control_mgr	control_rate_hz	250.0
	default_controller	px4_passthrough
trajectory_mgr	publish_rate_hz	50.0
	default_generator	waypoint_linear
uav_manager	takeoff_alt_m	2.0
safety_monitor	check_rate_hz	100.0
	max_radius_m	100.0
	max_altitude_m	50.0

Launch Hierarchy

- `simulation.launch.py` → `single_uav_sitl` → `single_uav`
- `multi_uav_sitl.launch.py` → $N \times \text{UAV}$ containers